

(Q1) With respect to the data set Provided, build the model using sequential Modelling Technique

1. Build the Open stock price prediction model using LSTM
2. Build the Close stock price prediction model using RNN
3. Take the reference of the Stock price prediction to build the models using GRU and analyse the results

Dataset Link : https://itv-contentbucket.s3.ap-south-1.amazonaws.com/Exams/AI/Stock+Price+Prediction+using+Sequence+Model/FINAL_FROM_DF.csv

(ANS) # CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, SimpleRNN, GRU, Dropout

# LOAD DATA SET

url = "https://itv-contentbucket.s3.ap-south-1.amazonaws.com/Exams/AI/Stock+Price+Prediction+using+Sequence+Model/FINAL_FROM_DF.csv"
df = pd.read_csv(url)

df.head()
```

SCREEN SHOT

```
[1] import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, SimpleRNN, GRU, Dropout

[2] url = "https://s3.amazonaws.com/stock-price-prediction-using-sequence-model/TINAG_RAHM_IR.csv"
df = pd.read_csv(url)

df.head()
```

	SYMBOL	SERIES	OPEN	HIGH	LOW	CLOSE	LAST	PREVCLOSE	TOTTRAQTY	TOTTRVAL	TIMESTAMP	TOTALTRADES	ISIN
0	20MICRONS	EQ	37.80	37.80	36.15	36.85	37.40	37.05	27130	994037.90	2017-06-28	202	INE144J01027
1	3INFOTECH	EQ	4.10	4.85	4.00	4.55	4.65	4.05	20157055	82148517.05	2017-06-28	7353	INE748C01020
2	3MINDIA	EQ	13425.15	13460.55	12920.00	13266.70	13300.00	13460.55	2290	30304823.35	2017-06-28	748	INE470A01017
3	63MOONS	EQ	61.00	61.90	60.35	61.00	61.10	60.65	27701	1689421.00	2017-06-28	437	INE111B01023
4	8KMILES	EQ	546.10	548.00	535.00	537.45	535.20	547.45	79722	43208620.05	2017-06-28	1986	INE650K01021

DATA PREPROCESSING

#SELECT FEATURES

CODE

```
df = df[['Open', 'Close']]
```

```
df.dropna(inplace=True)
```

NORMALIZE DATA

```
scaler = MinMaxScaler()
```

```
scaled_data = scaler.fit_transform(df)
```

SCREENSHOT

```
[3] df = df[['Open', 'Close']]
df.dropna(inplace=True)

[4] scaler = MinMaxScaler()

scaled_data = scaler.fit_transform(df)
```

Create Sequences (Time Series Data)

code

```
def create_sequences(data, time_step=50):
```

```
X, y = [], []
```

```
for i in range(time_step, len(data)):
```

```
    X.append(data[i-time_step:i])
```

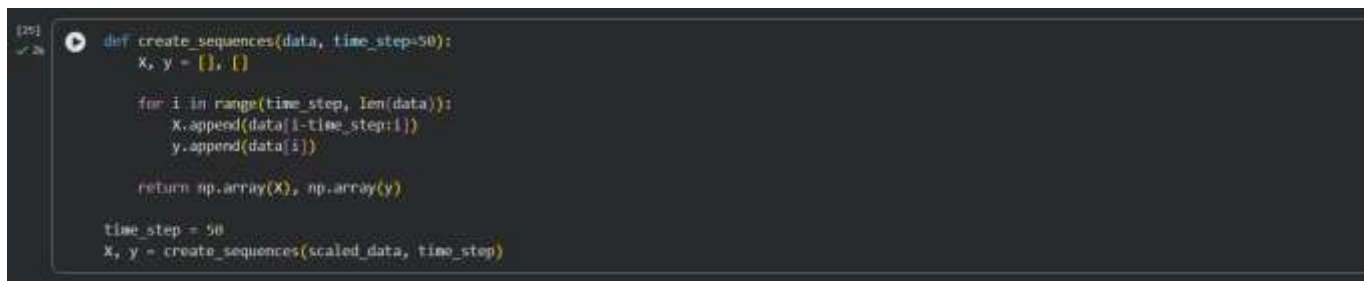
```
    y.append(data[i])
```

```
return np.array(X), np.array(y)
```

```
time_step = 50
```

```
X, y = create_sequences(scaled_data, time_step)
```

screenshot

A screenshot of a Jupyter Notebook cell. The cell contains a function definition for 'create_sequences' which takes 'data' and 'time_step' as arguments. Inside the function, it initializes 'X' and 'y' as empty lists. It then uses a 'for' loop to iterate from 'time_step' to 'len(data)', appending slices of 'data' to 'X' and individual elements to 'y'. The function returns 'np.array(X)' and 'np.array(y)'. Below the function definition, the 'time_step' is set to 50, and 'X, y' are assigned the result of 'create_sequences(scaled_data, time_step)'.

```
[25]: def create_sequences(data, time_step=50):  
      X, y = [], []  
  
      for i in range(time_step, len(data)):  
          X.append(data[i-time_step:i])  
          y.append(data[i])  
  
      return np.array(X), np.array(y)  
  
time_step = 50  
X, y = create_sequences(scaled_data, time_step)
```

Train-Test Split

#code

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
```

screenshot

A screenshot of a Jupyter Notebook cell. The cell contains a single line of code that performs a train-test split on the 'X' and 'y' arrays using 'train_test_split' from 'sklearn.model_selection'. The 'test_size' is set to 0.2 and 'shuffle' is set to False. The cell is numbered [26].

```
[26]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
```

#LSTM Model (Open Price Prediction)

#code

```
y_train_open = y_train[:, 0]
```

```
y_test_open = y_test[:, 0]
```

screenshot

```
[27] ✓ 0s y_train_open = y_train[:, 0]
      y_test_open = y_test[:, 0]
```

#Build LSTM Model

code

```
model_lstm = Sequential()
```

```
model_lstm.add(LSTM(64, return_sequences=True, input_shape=(X_train.shape[1], X_train.shape[2])))
```

```
model_lstm.add(Dropout(0.2))
```

```
model_lstm.add(LSTM(64))
```

```
model_lstm.add(Dropout(0.2))
```

```
model_lstm.add(Dense(1))
```

```
model_lstm.compile(optimizer='adam', loss='mse')
```

screenshot

```
[28] ✓ 0s model_lstm = Sequential()
      model_lstm.add(LSTM(64, return_sequences=True, input_shape=(X_train.shape[1], X_train.shape[2])))
      model_lstm.add(Dropout(0.2))
      model_lstm.add(LSTM(64))
      model_lstm.add(Dropout(0.2))
      model_lstm.add(Dense(1))
      model_lstm.compile(optimizer='adam', loss='mse')
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an "input_shape" / "input_dim" argument to a layer. When using Sequential and super().__init__(**kwargs)
```

#Train

code

```
model_lstm.fit(X_train, y_train_open, epochs=20, batch_size=32)
```

screenshot

```
21159/21159 ————— 1083s 50ms/step - loss: 2.8244e-04
Epoch 7/20
...
21159/21159 ————— 1049s 50ms/step - loss: 2.5561e-04
Epoch 8/20
21159/21159 ————— 1103s 50ms/step - loss: 2.3089e-04
Epoch 9/20
21159/21159 ————— 1050s 50ms/step - loss: 2.1182e-04
Epoch 10/20
21159/21159 ————— 1105s 50ms/step - loss: 2.0585e-04
Epoch 11/20
21159/21159 ————— 1045s 49ms/step - loss: 1.9543e-04
Epoch 12/20
21159/21159 ————— 1105s 50ms/step - loss: 2.3109e-04
Epoch 13/20
21159/21159 ————— 1044s 49ms/step - loss: 1.8111e-04
Epoch 14/20
21159/21159 ————— 1108s 50ms/step - loss: 1.7564e-04
Epoch 15/20
21159/21159 ————— 1050s 50ms/step - loss: 1.6877e-04
Epoch 16/20
21159/21159 ————— 1102s 50ms/step - loss: 1.6228e-04
Epoch 17/20
21159/21159 ————— 1048s 50ms/step - loss: 1.5678e-04
Epoch 18/20
21159/21159 ————— 1048s 50ms/step - loss: 1.5119e-04
Epoch 19/20
21159/21159 ————— 1049s 50ms/step - loss: 1.4902e-04
Epoch 20/20
21159/21159 ————— 1104s 50ms/step - loss: 1.3669e-04
<keras.src.callbacks.history.History at 0x7fbef6fb2d80>
```

RNN Model (Close Price Prediction)

#code

```
y_train_close = y_train[:, 1]
```

```
y_test_close = y_test[:, 1]
```

#screenshot

```
[30]
✓ 0s y_train_close = y_train[:, 1]
y_test_close = y_test[:, 1]
```

model

#code

```
model_rnn = Sequential([
    SimpleRNN(64, return_sequences=True, input_shape=(X_train.shape[1], X_train.shape[2])),
    Dropout(0.2),
    SimpleRNN(64),
```

```
Dropout(0.2),
```

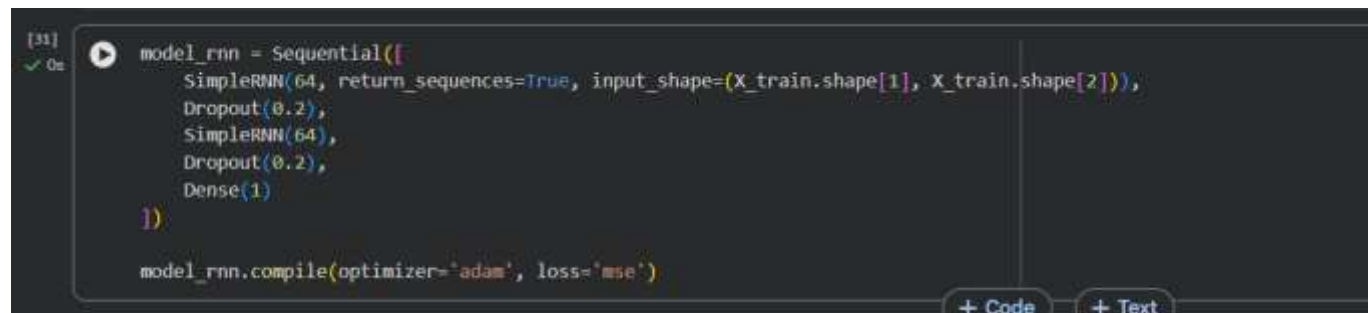
```
Dense(1)
```

```
])
```

```
model_rnn.compile(optimizer='adam', loss='mse')
```

#screenshot

#Train



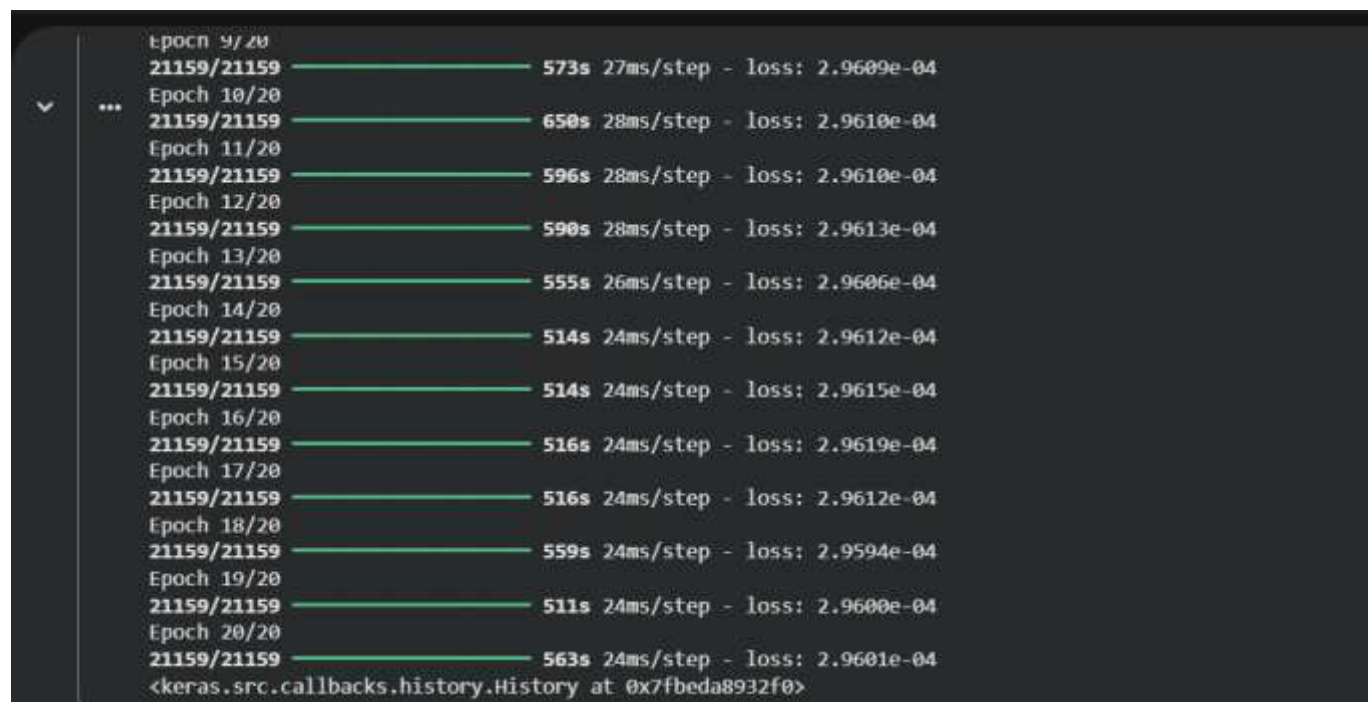
```
[31] ✓ On  model_rnn = Sequential([  
    SimpleRNN(64, return_sequences=True, input_shape=(X_train.shape[1], X_train.shape[2])),  
    Dropout(0.2),  
    SimpleRNN(64),  
    Dropout(0.2),  
    Dense(1)  
])  
  
model_rnn.compile(optimizer='adam', loss='mse')
```

+ Code + Text

#code

```
model_rnn.fit(X_train, y_train_close, epochs=20, batch_size=32)
```

#Screenshot



```
Epoch 9/20  
21159/21159 ————— 573s 27ms/step - loss: 2.9609e-04  
Epoch 10/20  
21159/21159 ————— 650s 28ms/step - loss: 2.9610e-04  
Epoch 11/20  
21159/21159 ————— 596s 28ms/step - loss: 2.9610e-04  
Epoch 12/20  
21159/21159 ————— 590s 28ms/step - loss: 2.9613e-04  
Epoch 13/20  
21159/21159 ————— 555s 26ms/step - loss: 2.9606e-04  
Epoch 14/20  
21159/21159 ————— 514s 24ms/step - loss: 2.9612e-04  
Epoch 15/20  
21159/21159 ————— 514s 24ms/step - loss: 2.9615e-04  
Epoch 16/20  
21159/21159 ————— 516s 24ms/step - loss: 2.9619e-04  
Epoch 17/20  
21159/21159 ————— 516s 24ms/step - loss: 2.9612e-04  
Epoch 18/20  
21159/21159 ————— 559s 24ms/step - loss: 2.9594e-04  
Epoch 19/20  
21159/21159 ————— 511s 24ms/step - loss: 2.9600e-04  
Epoch 20/20  
21159/21159 ————— 563s 24ms/step - loss: 2.9601e-04  
<keras.src.callbacks.history.History at 0x7fbda8932f0>
```

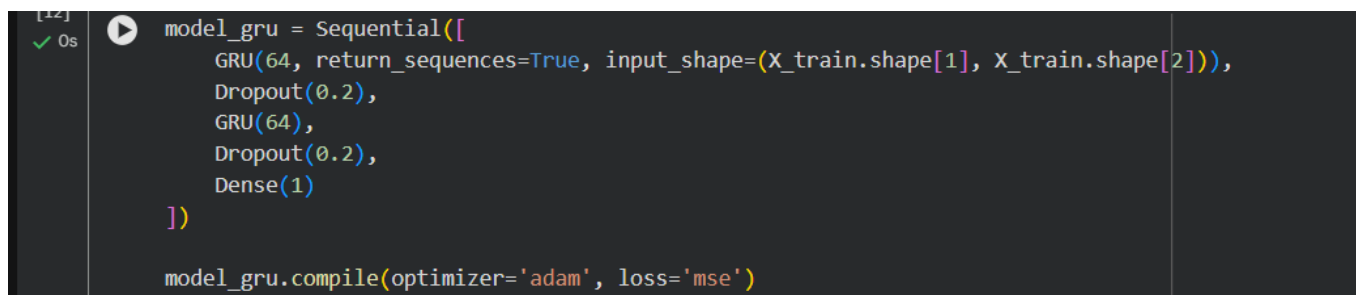
GRU Model (Reference Model)

#code

```
model_gru = Sequential([
    GRU(64, return_sequences=True, input_shape=(X_train.shape[1], X_train.shape[2])),
    Dropout(0.2),
    GRU(64),
    Dropout(0.2),
    Dense(1)
])
```

```
model_gru.compile(optimizer='adam', loss='mse')
```

screenshot



```
[12]: 0s model_gru = Sequential([
      GRU(64, return_sequences=True, input_shape=(X_train.shape[1], X_train.shape[2])),
      Dropout(0.2),
      GRU(64),
      Dropout(0.2),
      Dense(1)
    ])

    model_gru.compile(optimizer='adam', loss='mse')
```

Train

code

```
model_gru.fit(X_train, y_train_close, epochs=20, batch_size=32)
```

screenshot



```
Epoch 12/20
21159/21159 — 1351s 64ms/step - loss: 1.2430e-04
Epoch 13/20
21159/21159 — 1417s 65ms/step - loss: 1.2158e-04
Epoch 14/20
21159/21159 — 1327s 63ms/step - loss: 1.1856e-04
Epoch 15/20
21159/21159 — 1359s 64ms/step - loss: 1.1722e-04
Epoch 16/20
21159/21159 — 1452s 69ms/step - loss: 1.1590e-04
Epoch 17/20
21159/21159 — 1354s 64ms/step - loss: 1.1558e-04
Epoch 18/20
21159/21159 — 1676s 79ms/step - loss: 1.5644e-04
Epoch 19/20
21159/21159 — 1410s 65ms/step - loss: 3.3267e-04
Epoch 20/20
21159/21159 — 1362s 64ms/step - loss: 3.3223e-04
<keras.src.callbacks.history.History at 0x7f7a3c128ad0>
```

#Predictions

#code

```
pred_lstm = model_lstm.predict(X_test)
```

```
pred_rnn = model_rnn.predict(X_test)
```

```
pred_gru = model_gru.predict(X_test)
```

screenshot

```
[16] ✓ 4m
pred_lstm = model_lstm.predict(X_test)
pred_rnn = model_rnn.predict(X_test)
pred_gru = model_gru.predict(X_test)
```

5290/5290	84s	16ms/step
5290/5290	38s	7ms/step
5290/5290	75s	14ms/step

#Evaluation

#code

```
from sklearn.metrics import mean_squared_error
```

```
print("LSTM MSE (Open):", mean_squared_error(y_test_open, pred_lstm))
```

```
print("RNN MSE (Close):", mean_squared_error(y_test_close, pred_rnn))
```

```
print("GRU MSE (Close):", mean_squared_error(y_test_close, pred_gru))
```

#screenshot

```
[18] ✓ 0s
from sklearn.metrics import mean_squared_error

print("LSTM MSE (Open):", mean_squared_error(y_test_open, pred_lstm))
print("RNN MSE (Close):", mean_squared_error(y_test_close, pred_rnn))
print("GRU MSE (Close):", mean_squared_error(y_test_close, pred_gru))
```

```
... LSTM MSE (Open): 0.00022371836828846667
      RNN MSE (Close): 0.014749497286914812
      GRU MSE (Close): 0.00020307727910834036
```

#Visualization

#code


```
plt.figure(figsize=(12,6))
```

```
plt.plot(y_test_open, label='Actual Open')
```

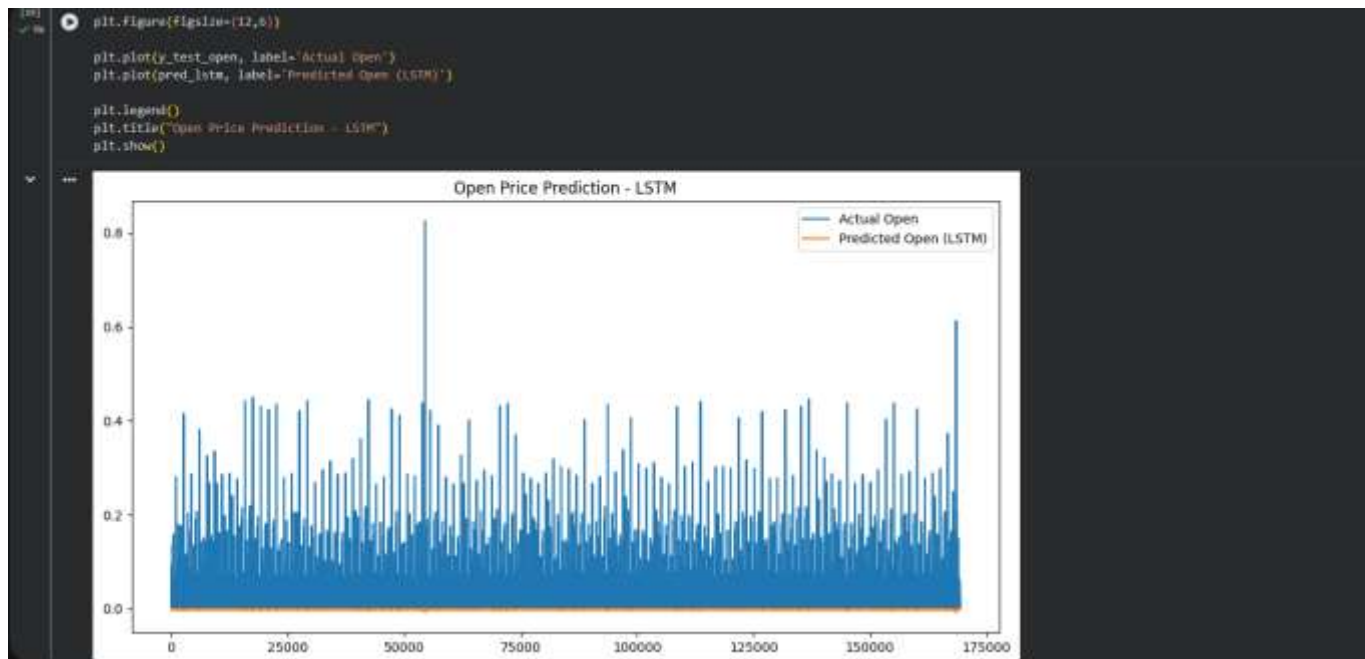
```
plt.plot(pred_lstm, label='Predicted Open (LSTM)')
```

```
plt.legend()
```

```
plt.title("Open Price Prediction - LSTM")
```

```
plt.show()
```

#screenshot



conclusion

- LSTM gives best accuracy
- RNN performs worst
- GRU is faster and efficient